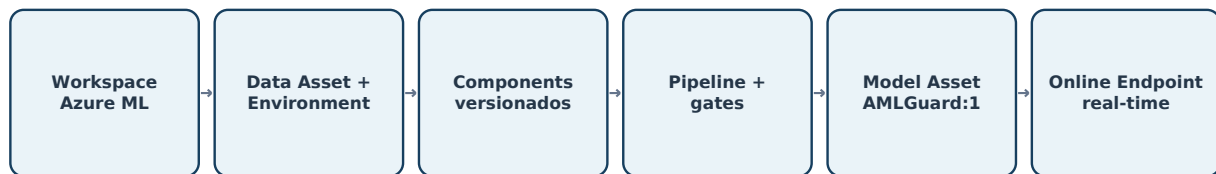


AMLGuard

Manual de Estudo - Volume 5

Azure Machine Learning: do pipeline em cloud ao endpoint em tempo real



Objetivo deste volume

Explicar, em linguagem simples e conectada ao projeto real, como o AMLGuard saiu de um MLOps local bem estruturado para uma arquitetura de treinamento, validação, registro e inferência executada no Azure Machine Learning.

Este volume cobre as entregas dos Dias 15 a 19: fundação do workspace, Data Asset e Environment, componentes reutilizáveis, pipeline end-to-end com quality gate e regression gate, registro controlado do modelo no Azure ML Model Registry e publicação em Managed Online Endpoint.

Mensagem central: cloud não é apenas "rodar o mesmo código em outro computador". É transformar dados, ambiente, treino, avaliação, promoção e serving em recursos versionados, auditáveis e reproduzíveis.

Versão do modelo em serving: Azure ML Model Asset AMLGuard:1 | Threshold público: 0.892163 | Endpoint: amlguard-realtime

1. Dias 15 e 16 - fundação da nuvem: workspace, dados e ambiente

Até o Dia 14, o AMLGuard já tinha testes, Docker, CI/CD e MLflow. O passo seguinte foi criar uma "fábrica na nuvem" onde os mesmos artefatos pudessem ser executados com infraestrutura gerenciada.

Analogia do mundo real - montar a fábrica antes de ligar a linha

Imagine abrir uma indústria. Antes de produzir qualquer peça, você precisa do terreno, do prédio, do almoxarifado e das máquinas com especificação conhecida. No Azure ML, o Workspace é o prédio, o Data Asset é o lote de matéria-prima catalogado e o Environment é a ficha técnica das máquinas e ferramentas usadas na produção.

Dia 15 - Azure ML Workspace

Recurso	Implementação no AMLGuard	Por que importa
Resource Group	rg-amlguard-dev-brazilsouth	Agrupa os recursos e facilita custo, governança e limpeza.
Workspace	mlw-amlguard-dev	Ponto central para jobs, assets, models e endpoints.
Região	brazilsouth	Mantém a fase cloud concentrada na mesma região.
Identidade	Managed Identity	Evita depender de credenciais embutidas no código.

Dia 16 - Data Asset e Environment

Asset	Versão	Conteúdo
ibm-aml-hi-small	1	Dataset HI-Small registrado como uri_file.
amlguard-training	1	Python 3.12 + pandas, numpy, scikit-learn, imbalanced-learn, XGBoost e joblib.

O dataset local foi validado por tamanho e SHA256 antes do registro. Isso cria uma ponte auditável entre o arquivo conhecido no computador e o asset usado na nuvem.

Por que usar serverless compute?

Nos Dias 16 a 18, os jobs usam Azure ML serverless compute. A infraestrutura é criada sob demanda para o job e liberada depois, evitando manter um cluster persistente apenas para um portfólio de estudo.

Erro real: os dois primeiros smoke tests falharam com ModuleNotFoundError: No module named src. A causa não era falta do código: executar python cloud/azure/jobs/smoke_test.py alterava a forma como o Python montava o import path. A correção foi executar como módulo: python -m cloud.azure.jobs.smoke_test.

Autoteste: 1) Qual a diferença entre um Data Asset e simplesmente apontar para um arquivo local? 2) Por que um Environment versionado ajuda a reproduzir um treino meses depois? 3) O que mudou ao executar o smoke test com -m?

2. Dia 17 - componentes reutilizáveis: preparar, treinar e avaliar

Com dados e ambiente registrados, o pipeline foi dividido em três componentes independentes. A ideia é separar responsabilidades: cada componente recebe entradas bem definidas, produz saídas claras e pode ser validado isoladamente.

Analogia do mundo real - três estações da linha de produção

Uma fábrica profissional não mistura corte, montagem e inspeção na mesma bancada. Cada estação faz uma função, recebe uma peça padronizada e entrega outra peça padronizada para a estação seguinte. Isso facilita testar, substituir e versionar etapas sem reescrever a fábrica inteira.

Componente	Entrada principal	Saída principal	Responsabilidade
amlguard_prepare_data:1	IBM AML Data Asset	prepared_split.joblib	Validar dados, construir features leakage-safe e produzir split preparado.
amlguard_train_model:1	prepared_split.joblib	model.joblib + metadata	Treinar o Weighted XGBoost congelado.
amlguard_evaluate_model:1	prepared data + model	metrics.json	Calcular métricas e aplicar quality/regression gates.

Por que testar cada componente isoladamente?

Antes de encadear tudo, cada estação foi executada sozinha. Isso reduz o espaço de busca de erros: se o pipeline completo falhar depois, já sabemos que os componentes individuais funcionavam e podemos investigar ligação de inputs/outputs, permissões ou configuração do parent job.

Etapas	Run ID	Status
prepare_data	lucid_bag_rblj448q2k	Completed
train_model	nice_map_jw7r2gwc3k	Completed
evaluate_model	helpful_snake_mk7qwbycm5	Completed

O componente de avaliação já implementava duas camadas de proteção: quality gate (mínimos absolutos) e regression gate (comparação contra o baseline congelado). O modelo não é aprovado apenas porque o job terminou sem erro.

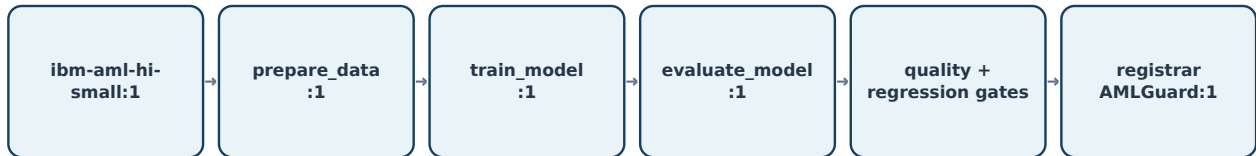
Lição central: Completed significa que o software executou. PASS significa que o resultado atende às regras de qualidade definidas pelo projeto. São conceitos diferentes.

Autoteste: 1) Qual vantagem de versionar componentes? 2) Por que o componente de treino não deve registrar automaticamente o modelo antes da avaliação? 3) O que você investigaria primeiro se os três componentes isolados funcionassem, mas o pipeline encadeado falhasse?

3. Dia 18 - pipeline end-to-end, gates e promoção controlada

O Dia 18 juntou os três componentes em um único Azure ML Pipeline. O objetivo deixou de ser testar peças separadas e passou a provar o fluxo completo: dados versionados entram, modelo é treinado, avaliação é executada e só então ocorre a decisão de promoção.

Fluxo implementado



Estratégia de promoção

O registro do modelo não foi embutido no componente de treino. O fluxo deliberado foi: treinar -> avaliar -> comprovar gates -> registrar. Isso evita que um modelo tecnicamente treinado, mas degradado, vire automaticamente um asset aprovado.

Métrica / gate	Resultado observado	Regra
Average Precision	0.0368334059875	≥ 0.035
Recall no operating point	0.6703155184	≥ 0.65 com precision $\geq 2\%$
Quality gate	PASS	CrITÉrios mÍnimos absolutos
Regression gate	PASS	Sem regressões fora das tolerâncias
Threshold	0.8921634555	Paridade com threshold público congelado

O pipeline parent wheat_cassava_65j58pbzkl terminou como Completed; os três child jobs também terminaram como Completed. O metrics.json foi baixado e verificado antes do registro.

Depois da aprovação, foi criado o Azure ML Model Asset AMLGuard:1, do tipo custom_model, associado ao job do pipeline.

Distinção importante: o MLflow Registered Model Version 1 dos Dias 13-14 e o Azure ML Model Asset AMLGuard:1 do Dia 18 são registros em sistemas diferentes. O número "1" coincide, mas os registries não são o mesmo objeto.

Autoteste: 1) Por que "pipeline Completed" não basta para registrar o modelo? 2) O que o regression gate protege? 3) Qual a diferença entre o MLflow Registry local e o Azure ML Model Registry usado nesta fase?

4. Dia 19 - Managed Online Endpoint e inferência em tempo real

Depois de registrar o modelo aprovado, o passo seguinte foi disponibilizá-lo para requisições online. O AMLGuard passou a ter um endpoint gerenciado pelo Azure ML, com autenticação por chave, ambiente de inferência dedicado e um deployment chamado blue.

Analogia do mundo real - da fábrica para o balcão de atendimento

O pipeline cloud é a fábrica: produz e inspeciona o modelo. O Model Registry é o estoque de versões aprovadas. O Online Endpoint é o balcão aberto ao cliente: recebe um pedido padronizado, escolhe a versão implantada e devolve uma resposta em segundos.

Recurso	Valor no AMLGuard	Função
Endpoint	amlguard-realtime	Endereço lógico do serviço de inferência.
Auth mode	key	Protege chamadas por autenticação de chave.
Deployment	blue	Implantação ativa que recebe o tráfego.
Traffic	blue = 100%	Todo o tráfego foi roteado para a implantação validada.
Model	AMLGuard:1	Asset aprovado no Dia 18.
Environment	amlguard-inference:1	Dependências específicas de serving.
Instance	Standard_DS2_v2 x 1	Compute persistente enquanto o deployment estiver ativo.

Por que um ambiente de inferência separado?

Treino e serving têm necessidades diferentes. O ambiente de treino inclui tudo para preparar dados e ajustar modelos; o de inferência precisa apenas do necessário para carregar o artefato, receber JSON e calcular predict_proba. Separar reduz acoplamento e deixa a intenção de cada ambiente explícita.

Contrato de resposta cloud

Campo	Exemplo validado	Significado
score	0.99112468957901	Score de risco retornado pelo XGBoost.
alert	true	Verdadeiro quando score >= threshold.
threshold	0.892163	Threshold público congelado.
model_version	"1"	Versão do Azure ML Model Asset em serving.

Lição central: implantar um modelo é diferente de publicá-lo em registry. Registry responde "qual versão existe/aprovada?"; endpoint responde "como um sistema externo usa essa versão agora?".

Autoteste: 1) Qual a diferença entre endpoint e deployment? 2) Por que 100% de tráfego no blue é útil para validar uma primeira implantação? 3) Por que model_version = "1" não deve ser confundido com a versão de aplicação 0.1.0 usada anteriormente?

5. Erros reais da fase cloud - o que eles ensinam

Os erros mais valiosos de um projeto de portfólio são os que geram entendimento operacional. Esta fase produziu problemas reais de imports, providers do Azure, estados de recursos e compatibilidade entre Windows, Python e Azure CLI.

Erro real	Causa	Correção / lição
ModuleNotFoundError: src	Script executado por caminho de subpasta alterava o import path.	Executar como módulo com python -m. Contexto de execução importa.
SubscriptionNotRegistered	Providers necessários ao Managed Online Endpoint não estavam registrados.	Registrar Microsoft.Network, Microsoft.PolicyInsights e Microsoft.Cdn antes de nova tentativa.
Endpoint ficou Failed	Primeira criação ocorreu enquanto providers ainda estavam indisponíveis.	Apagar o recurso Failed e recriar limpo após pré-requisitos estarem Registered.
Python não encontrava az	Azure CLI funcionava no CMD, mas subprocess não localizava o executável.	No helper de evidência, chamar o Azure CLI via cmd.exe /c az no Windows.
Ruff I001	Bloco de imports do score.py fora da ordenação esperada.	Usar ruff --fix e validar novamente antes do deploy.

Analogia do mundo real - instalação elétrica de uma loja

Você pode ter o equipamento perfeito, mas a loja não abre se a concessionária ainda não ativou o fornecimento, o quadro estiver em estado de falha ou a tomada usar outro padrão. Na cloud, providers, identidades, estados do recurso e ambiente de execução são parte do sistema - não são detalhes externos ao código.

Regra profissional desta fase

Não "consertar" o código quando a evidência aponta para infraestrutura. Primeiro classifique o problema: código, ambiente, dependência, autenticação, provider, quota, rede ou estado do recurso. Mudar o modelo ou os componentes v1 sem diagnóstico teria mascarado a causa e quebrado a rastreabilidade.

Autoteste: 1) Por que apagar e recriar um endpoint Failed pode ser melhor que tentar vários updates cegos? 2) O que distingue um erro de infraestrutura de um erro do scoring script? 3) Por que registrar providers é um requisito da assinatura, não do repositório?

6. Evidências, comandos essenciais e custo

Comandos essenciais da fase

Registrar environment

```
az ml environment create --file cloud\azure\online\environment.yml
```

Criar endpoint

```
az ml online-endpoint create --file cloud\azure\online\endpoint.yml
```

Criar deployment e enviar todo o tráfego

```
az ml online-deployment create --file cloud\azure\online\deployment.yml --all-traffic
```

Invocar o endpoint

```
az ml online-endpoint invoke --name amlguard-realtime --request-file cloud\azure\online\sample_request.json
```

Verificar tráfego

```
az ml online-endpoint show --name amlguard-realtime --query traffic --output json
```

Apagar o endpoint quando não for mais necessário

```
az ml online-endpoint delete --name amlguard-realtime --yes
```

Evidências preservadas no repositório

Arquivo	O que prova
docs/evidence/day16_smoke_summary.json	Import do projeto e validação de schema no ambiente cloud.
docs/evidence/day17_evaluation_metrics.json	Avaliação dos componentes isolados e gates.
docs/evidence/day18_evaluation_metrics.json	Pipeline end-to-end reproduziu baseline e passou gates antes do registro.
docs/evidence/day19_online_endpoint.json	Endpoint/deployment Succeeded, blue=100% e inferência real validada sem segredos.

Custo - diferença importante entre treino e serving

Os jobs de treinamento desta fase usaram serverless compute e não deixaram cluster persistente. Já o Managed Online Endpoint usa uma instância Standard_DS2_v2 enquanto o deployment está ativo. Isso significa custo contínuo de compute. Para um projeto de portfólio, faz sentido manter o endpoint apenas durante testes, demonstrações e etapas que dependem de tráfego real.

Lição central: otimização de custo também é MLOps. Um serviço que funciona tecnicamente, mas fica ligado sem necessidade, continua sendo uma decisão operacional ruim.

7. Definição de pronto dos Dias 15 a 19

Evidência	Status
Azure ML workspace criado com identidade gerenciada	Concluído
Data Asset e training Environment versionados	Concluído
Smoke test cloud validado	Concluído
3 componentes v1 registrados e testados isoladamente	Concluído
Pipeline end-to-end executado	Concluído
Quality gate PASS e regression gate PASS	Concluído
Azure ML Model Asset AMLGuard:1 registrado após os gates	Concluído
Inference Environment amlguard-inference:1 registrado	Concluído
Managed Online Endpoint criado com key auth	Concluído
Deployment blue Succeeded e 100% do tráfego	Concluído
Inferência real retornou score, alert, threshold e model_version	Concluído
Evidências sanitizadas e README/documentação atualizados	Concluído
56 testes e Ruff limpo antes do commit	Concluído

Resumo para entrevista

"No AMLGuard, levei o pipeline de um fluxo MLOps local para Azure Machine Learning. Registre dataset e environments versionados, decompondo o treino em componentes reutilizáveis de preparação, treinamento e avaliação. Depois encadeei esses componentes em um pipeline end-to-end com quality gate e regression gate. O modelo só foi registrado como Azure ML Model Asset depois de comprovar os gates. Em seguida, criei um Managed Online Endpoint com ambiente de inferência dedicado, key authentication e um deployment blue recebendo 100% do tráfego. Validei uma requisição real em cloud retornando score, alert, threshold e model version, e preservei evidências sanitizadas de cada etapa."

O que esta fase demonstra profissionalmente

O projeto passou a mostrar, de forma verificável, competências de Azure Machine Learning, MLOps, pipeline orchestration, reusable components, model governance, quality gates, regression testing, model registry, cloud deployment e real-time inference. Para recrutadores, o diferencial não é apenas conhecer essas palavras: é ter evidências concretas no repositório de que o fluxo foi executado e validado.

Checklist mental antes do próximo volume

O Dia 20 deve responder a uma nova pergunta: depois que o modelo está no ar, como sabemos se ele continua saudável? Isso abre o bloco de observabilidade: latência, erros, alert rate, distribuição de scores, drift e estratégia de retraining. Dependendo da profundidade implementada, esse conteúdo pode iniciar o Volume 6.

Autoteste final: 1) Conte o fluxo completo do Dia 15 ao 19 sem consultar o manual. 2) Em qual ponto o modelo recebe autorização para ser registrado? 3) Qual evidência prova que o endpoint realmente serviu o modelo? 4) Qual recurso desta fase gera custo contínuo enquanto permanece ativo?